API Specification — Leish! Aesthetic Marketplace

Version: 1.0 · **Date:** 2026-10-12 · **Base path:** `/api`

Contract rules

All requests and responses are JSON unless a media endpoint explicitly says otherwise. Protected requests use `Authorization: Bearer <JWT>`. Validation errors use HTTP 400, missing/invalid authentication uses 401, authenticated but unauthorized access uses 403, missing resources use 404, and unexpected failures use 500 with a safe message. Error responses must not include stack traces, secrets or password data.

Authentication endpoints

Method	Path	Auth	Purpose
POST	`/auth/register`	none	Create client/provider account; returns sanitized user and JWT
POST	`/auth/login`	none	Verify credentials; returns sanitized user and JWT
GET	`/auth/me`	JWT	Return current authenticated user
PUT	`/auth/profile`	JWT	Update the caller's own profile
GET	`/auth/demo-accounts`	restricted	Return only approved demo metadata, or remove endpoint

Example login response:


```
{ "user": { "id": "user-1", "name": "Demo Client", "email": "client@example.test", "role": "client" }, "token": "<jwt>" }
```


Catalog endpoints

Method	Path	Auth	Purpose
GET	`/salons`	none	List public catalog
GET	`/salons/:id`	none	Read public listing details
PUT	`/salons/:id`	provider/admin	Update owned listing
POST	`/salons/:id/services`	provider/admin	Add service to owned listing
DELETE	`/salons/:id/services/:serviceId`	provider/admin	Delete service from owned listing
GET	`/reviews/:salonId`	none	List public reviews
POST	`/reviews`	client	Create review for eligible completed booking

Booking endpoints

Method	Path	Auth	Purpose
GET	`/bookings?email=...`	JWT	Client-scoped appointment list
GET	`/bookings?salonId=...`	JWT	Provider-owned appointment list
POST	`/bookings`	none or JWT	Create a validated `pending` booking request
PATCH	`/bookings/:id/status`	JWT	Cancel by owning client or transition by owning provider/admin
POST	`/venue/estimate-travel`	none	Estimate travel distance and surcharge; current implementation i

Minimum booking request:


```
{
  "salonId": "salon-1",
  "serviceId": "service-1",
  "date": "2026-10-20",
  "time": "10:00",
  "clientName": "Example Client",
  "clientEmail": "client@example.test",
  "clientPhone": "+60 12 000 0000"
}
```

Gemini endpoints

Method	Path	Result envelope
POST	`/gemini/advice`	`{ source, data }`
POST	`/gemini/summarize-reviews`	`{ source, data: { vibe, bestFor, tip } }`
POST	`/gemini/trends-lookbook`	`{ source, data: LookbookItem }`
POST	`/gemini/quality-audit`	`{ source, data: audit }`
POST	`/gemini/chat`	`{ source, data: message }`
POST	`/gemini/generate-image`	image result or labeled fallback
POST	`/gemini/generate-video`	async operation
POST	`/gemini/video-status`	operation status
POST	`/gemini/video-download`	`video/mp4` stream

All user-facing AI endpoints require zod input validation, a wall-clock timeout, parsed model output, a fallback path, and an AI-specific rate-limit bucket.

Authorization matrix

Operation	Guest	Client own	Provider own	Admin
Read public catalog	yes	yes	yes	yes
Create booking	yes	yes	yes	yes
Read client bookings	no	own only	no	policy-defined
Read provider bookings	no	no	own salon only	yes
Cancel client booking	no	own only	no	policy-defined
Transition provider booking	no	no	own salon only	yes
Modify service/profile	no	no	own salon only	yes
Create review	no	eligible booking only	no	moderation policy

Compatibility and versioning

The current API is unversioned. Breaking changes require a new `/api/v2` prefix or an explicitly documented migration. Additive response fields are preferred; clients must tolerate unknown fields. Every contract change must update this document, shared schemas and integration tests.